

Machine Learning in Variational Monte Carlo

William Bjørge Banelind, Emily Ania Puchala, Jakob Wiig Ryther, Aleksander Sekkelsten
(Dated: June 1, 2025)

Representing a wave function with a **Restricted Boltzmann Machine (RBM)**, was recently introduced, where G. Carleo and M Troyer (2017) applied a **Neural Network Quantum State (NQS)** to a quantum mechanical spin lattice system of the Ising and Heisenberg models [1]. To test additional applications of the **RBM** we applied it to a system of non-interacting, and interacting particles confined in a harmonic oscillator trap. To estimate the ground state of our system we trained our **RBM** using **Variational Monte Carlo (VMC)**. We tested varying parameters and found the following best parameters for the non-interacting 1 particle in 1 dimension case; learning rate = 0.1, stochastic gradient descent optimizer, metropolis algorithm with importance sampling, and 5 hidden units. We performed a statistical analysis using the blocking method that validated our parameters resulting in a mean energy $E = 0.49999 \pm 0.000015$ giving good agreement with the expected energy for our system. Expanding this to an interacting case with 2 particles in 2 dimensions we found similar parameters worked well, although the learning rate needed to be adjusted. In these systems our results agreed with the expected analytical results. We replaced the **RBM** with a **Neural Network (NN)**, and compared our findings where we highlighted the advantages and disadvantages of each method.

I. INTRODUCTION

The field of machine learning, specifically deep learning, has many applications in physics, with one of those being the use of **NNs** to study quantum many-body systems. One example of this is a **NQS**, which uses a **NN** to approximate wave functions of many-body systems. Representing a wave function with a **RBM** was introduced by Carleo and Troyer in 2017 where they applied it to the Ising model and Heisenberg model [1].

An **RBM** is a type of **Boltzmann Machine (BM)** where, unlike the **BM**, connections between hidden units and visible units in the same layer are not allowed. This means the units in the same layer are not connected to each other, giving rise to the name “restricted.” These layer connections in an **RBM** are shown in FIG. 1. Typically **RBM**s have one hidden layer, although deeper architectures can be constructed by stacking multiple **RBM**s.

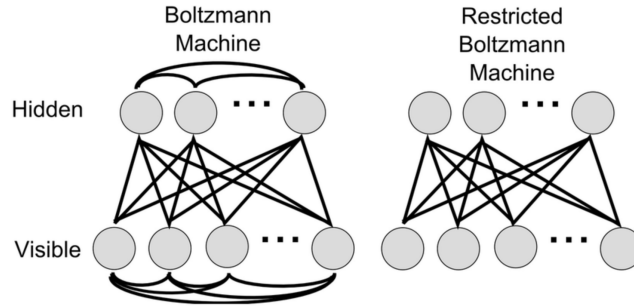


FIG. 1: A visual representation of a **BM** (left) and **RBM** (right), where the unit connections are shown as connected to the same layer in the **BM** but not in the **RBM**. Hidden and visible layers are labeled accordingly. Figure sourced from O’Connor et. al. (2017) [2].

In this study, we apply an **RBM** to a system of interacting particles confined in a harmonic trap. The **RBM** serves as our primary deep learning method, although this can be replaced with a standard **NN**. We represent our wave equation as an **RBM** and evaluate it using both standard metropolis sampling and importance sampling. Furthermore, we perform a statistical analysis making use of the blocking method to validate our results. After initially starting with a non-interacting case, we later evaluate the same system taking into consideration interactions between particles. We compare the results when the **RBM** is replaced with a **NN** in the context of quantum many-body problems. We will study how we can apply the **RBM** to quantum mechanical systems and test its applicability to a quantum many-body problem.

In Section II we describe the mathematical background and in Section III we describe the methods and algorithm which can be implemented in any programming language. Section IV contains the results and a discussion of the analysis of the results. A summary of the methods and results can be found in Section V. All code developed for this

report is available via our GitHub repository,¹ which includes a README.md file that outlines the structure of the code.

II. THEORY

A. Quantum mechanics

We consider a system of P particles confined in a pure two-dimensional isotropic harmonic oscillator potential, with pairwise Coulomb repulsion. In atomic units ($\hbar = c = e = m_e = 1$), the Hamiltonian is

$$\hat{H} = \sum_{i=1}^P \left(-\frac{1}{2} \nabla_i^2 + \frac{1}{2} \omega^2 r_i^2 \right) + \sum_{i < j} \frac{1}{r_{ij}} \quad (1)$$

where $r_i = |\mathbf{r}_i| = \sqrt{x_i^2 + y_i^2}$ and $r_{ij} = |\mathbf{r}_i - \mathbf{r}_j|$. For concreteness, we focus on $P = 2$ particles with $\bar{\omega} = 1$. In that case, the exact interacting two-particle ground-state energy is known from Taut (1993) to be $E_{\text{gs}} = 3$ a.u. [3].

Although this ground state originally describes two interacting electrons (fermions), the Coulomb Hamiltonian $\hat{H}$ in Eq. (1) does not act on spin. Hence, to simplify our implementation in terms of symmetric ansätze, we treat both particles as bosons occupying the symmetric spatial ground state

$$\Phi(\mathbf{r}_i, \mathbf{r}_j) = C \exp \left[-\frac{1}{2} (r_1^2 + r_2^2) \right], \quad (2)$$

where $\omega = 1$ has been substituted, and the normalization constant C is fixed by $\int |\Phi|^2 d\mathbf{r}_1 d\mathbf{r}_2$ (in fact, $C = 1/\pi$). Since $\hat{H}$ in Eq. (1) acts only on spatial coordinates, the energy expectation $\langle \Phi | \hat{H} | \Phi \rangle$ remains 3 a.u. even if Φ is interpreted as a bosonic wave function.

If the Coulomb term is omitted, the non-interacting single-particle ground state in 2D has energy ω , so two non-interacting particles have total energy $2\omega = 2$ a.u. When interactions are restored the ground-state energy increases to 3 a.u. Hence our study will compare how well a neural-network ansatz (**RBM** and **NN**) augmented with a Jastrow factor can recover $E = 2$ a.u. in the non-interacting limit and $E = 3$ a.u. in the interacting case.

B. Restricted Boltzmann Machine

We represent our variational wave function $\Psi(\mathbf{X})$ using a Gaussian-binary **RBM**. Let $\mathbf{X} = (X_1, X_2, \dots, X_M)^\top \in \mathbb{R}^M$ be the vector of visible units, where $M = 2P$ (the x, y coordinates of P particles), and let $\mathbf{H} = (H_1, H_2, \dots, H_N)^\top \in \{0, 1\}^N$ be the binary hidden units. We define the “physics-convention” energy function,

$$E(\mathbf{X}, \mathbf{H}) = -\sum_{j=1}^N b_j H_j - \frac{1}{\sigma^2} \sum_{i=1}^M \sum_{j=1}^N X_i W_{ij} H_j - \sum_{i=1}^M \frac{(X_i - a_i)^2}{2\sigma^2}, \quad (3)$$

where $\{a_i\}_{i=1}^M$ are visible-unit biases, $\{b_j\}_{j=1}^N$ are hidden-unit biases, W_{ij} are the visible-hidden weights, and $\sigma > 0$ is the shared Gaussian width. The unnormalized joint density is

$$\Psi(\mathbf{X}, \mathbf{H}) = \exp[-E(\mathbf{X}, \mathbf{H})],$$

and the partition function is

$$Z = \int_{\mathbb{R}^M} \sum_{\mathbf{H} \in \{0, 1\}^N} \exp[-E(\mathbf{X}, \mathbf{H})] d\mathbf{X}.$$

Marginalizing over $\mathbf{H}$ yields the **RBM** wave function on the visible layer:

$$\begin{aligned} \Psi_{\text{RBM}}(\mathbf{X}) &= \sum_{\mathbf{H} \in \{0, 1\}^N} \exp[-E(\mathbf{X}, \mathbf{H})] \\ &= \exp \left[-\sum_{i=1}^M \frac{(X_i - a_i)^2}{2\sigma^2} \right] \prod_{j=1}^N \left(1 + e^{b_j + \frac{1}{\sigma^2} \sum_{i=1}^M X_i W_{ij}} \right). \end{aligned} \quad (4)$$

¹ <https://github.uio.no/jakobwr/FYS4411-VMCML>

We then define the normalized variational ansatz

$$\Psi(\mathbf{X}) = \frac{1}{\sqrt{Z}} \Psi_{\text{RBM}}(\mathbf{X}),$$

but since Z is a constant with respect to the coordinates, it does not appear in any Monte Carlo ratio or gradient.

The variational parameters are $\{\alpha_k\} = \{a_1, \dots, a_M, b_1, \dots, b_N, W_{11}, \dots, W_{MN}, \sigma\}$. To optimize them, we minimize the expectation value of the local energy

$$E_L(\mathbf{X}) = \frac{1}{\Psi(\mathbf{X})} \hat{H} \Psi(\mathbf{X}). \quad (5)$$

The variational parameters $\{\alpha_k\}$ are then updated by minimizing $\langle E_L \rangle$ via stochastic gradient descent on $|\Psi|^2$.

1. Gradients of $\ln \Psi$

It is convenient to work with $\ln \Psi(\mathbf{X})$. From Eq. (4),

$$\ln \Psi(\mathbf{X}) = - \sum_{i=1}^M \frac{(X_i - a_i)^2}{2\sigma^2} + \sum_{j=1}^N \ln \left(1 + e^{b_j + \frac{1}{\sigma^2} \sum_{i=1}^M X_i W_{ij}} \right). \quad (6)$$

Define

$$\arg_j(\mathbf{X}) = b_j + \frac{1}{\sigma^2} \sum_{i=1}^M X_i W_{ij}, \quad t_j(\mathbf{X}) = \sigma(\arg_j(\mathbf{X})) = \frac{1}{1 + e^{-\arg_j(\mathbf{X})}}.$$

Then the partial derivatives of $\ln \Psi$ with respect to the variational parameters are

$$\frac{\partial \ln \Psi}{\partial a_m} = \frac{X_m - a_m}{\sigma^2}, \quad (7)$$

$$\frac{\partial \ln \Psi}{\partial b_n} = \frac{1}{1 + \exp[-\arg_n(\mathbf{X})]} = t_n(\mathbf{X}), \quad (8)$$

$$\frac{\partial \ln \Psi}{\partial W_{mn}} = \frac{X_m}{\sigma^2} \frac{1}{1 + \exp[-\arg_n(\mathbf{X})]} = \frac{X_m}{\sigma^2} t_n(\mathbf{X}), \quad (9)$$

$$\frac{\partial \ln \Psi}{\partial \sigma} = \sum_{i=1}^M \left[\frac{(X_i - a_i)^2}{\sigma^3} \right] - \frac{2}{\sigma^3} \sum_{j=1}^N (W_j \cdot \mathbf{X}) t_j(\mathbf{X}), \quad (10)$$

where $W_j \cdot \mathbf{X} = \sum_{i=1}^M X_i W_{ij}$. These expressions are used to compute the gradient of the cost function (see Sec. [IIB3](#))

2. Local Energy in Terms of $\nabla \ln \Psi$

From Eq. (5), the local energy can be written as

$$E_L(\mathbf{X}) = -\frac{1}{2} \sum_{i=1}^P \frac{\nabla_i^2 \Psi(\mathbf{X})}{\Psi(\mathbf{X})} + V_{\text{HO}}(\mathbf{X}) + V_{\text{ee}}(\mathbf{X}), \quad (11)$$

where $V_{\text{HO}} = \frac{1}{2} \sum_i \omega^2 r_i^2$ and $V_{\text{ee}} = \sum_{i < j} 1/r_{ij}$. Noting that

$$\frac{\nabla_i^2 \Psi}{\Psi} = \sum_{k=x,y} \left[\frac{\partial^2 \ln \Psi}{\partial x_{i,k}^2} + \left(\frac{\partial \ln \Psi}{\partial x_{i,k}} \right)^2 \right],$$

we see that evaluating E_L requires computing $\nabla_i \ln \Psi$ and $\nabla_i^2 \ln \Psi$, as detailed in Sec. [III](#).

3. Parameter Optimization Objective

We optimize the variational parameter $\{\alpha_k\}$ by minimizing the expectation value of the local energy,

$$\mathcal{L}(\{\alpha_k\}) = \langle E_L(\mathbf{X}) \rangle = \int d\mathbf{X} |\Psi(\mathbf{X})|^2 E_L(\mathbf{X}). \quad (12)$$

A short calculation shows that the gradient of $\mathcal{L}$ with respect to any parameter α_k is

$$\frac{\partial \mathcal{L}}{\partial \alpha_k} = 2 \left[\langle E_L O_{\alpha_k} \rangle - \langle E_L \rangle \langle O_{\alpha_k} \rangle \right], \quad O_{\alpha_k}(\mathbf{X}) = \frac{\partial \ln \Psi(\mathbf{X})}{\partial \alpha_k}. \quad (13)$$

Hence, in each Monte Carlo iteration, one samples $\mathbf{X} \sim |\Psi|^2$, computes $E_L(\mathbf{X})$ and $O_{\alpha_k}(\mathbf{X})$, estimates $\partial \mathcal{L} / \partial \alpha_k$ via Eq. (13), and updates $\alpha_k \rightarrow \alpha_k - \eta \partial \mathcal{L} / \partial \alpha_k$.

C. Jastrow Factor for Two-Particle Coulomb Interactions

When the Coulomb repulsion $\sum_{i<j} 1/r_{ij}$ is included, the exact wave function must satisfy the Kato cusp condition at $r_{ij} = 0$ [4]. For two particles interacting via

$$V_{ee}(r_{12}) = \frac{1}{r_{12}},$$

the cusp condition requires

$$\left. \frac{\partial \ln \Psi}{\partial r_{12}} \right|_{r_{12}=0} = \frac{1}{2}. \quad (14)$$

A simple way to enforce an approximate cusp is to introduce a multiplicative Jastrow factor:

$$\Psi_{\text{int}}(\mathbf{X}) = \Psi_{\text{RBM}}(\mathbf{X}) \times \exp \left[a_{\text{jastrow}} \sum_{i<j} \frac{1}{r_{ij}} \right], \quad (15)$$

where a_{jastrow} is a variational parameter. Near $r_{ij} = 0$, the factor $\exp[a_{\text{jastrow}}/r_{ij}]$ produces the required linear cusp in Ψ_{int} . Physically, this term captures short-range electron-electron correlation that the RBM alone cannot encode. In practice, once a_{jastrow} is optimized via VMC, the combined ansatz in Eq. (15) significantly improves the ground-state energy.

When $\Psi = \Psi_{\text{int}}$, the local energy in Eq. (11) acquires additional gradient and Laplacian terms due to $\ln \Psi_{\text{int}} = \ln \Psi_{\text{RBM}} + a_{\text{jastrow}} \sum_{i<j} 1/r_{ij}$. The explicit form of these derivatives is given in Sec. III.

D. Variational Monte Carlo

In VMC, one samples particle configurations $\mathbf{X}$ from the probability density $|\Psi(\mathbf{X})|^2$ using either Metropolis-Hastings or Langevin (importance-sampling) moves. At each configuration $\mathbf{X}^{(k)}$,

- compute the local energy $E_L(\mathbf{X}^{(k)})$ via Eq. (11), which requires evaluating $\nabla_i \ln \Psi$ and $\nabla_i^2 \ln \Psi$ (Sec. IIB 2);
- compute $O_{\alpha_k}(\mathbf{X}^{(k)}) = \partial_{\alpha_k} \ln \Psi$ for each variational parameter α_k (Sec. IIB 1).

The expectation values $\langle E_L \rangle$, $\langle O_{\alpha_k} \rangle$, and $\langle E_L O_{\alpha_k} \rangle$ are estimated by averaging over N_{MC} sampled configurations. The gradient of the cost function is then given by Eq. (13), and parameters are updated via

$$\alpha_k \mapsto \alpha_k - \eta \frac{\partial \mathcal{L}}{\partial \alpha_k},$$

where η is the learning rate.

a. Importance sampling. When using Langevin moves, one defines the quantum force

$$\mathbf{F}_i(\mathbf{X}) = 2\nabla_i \ln \Psi(\mathbf{X}), \quad (16)$$

and proposes

$$\mathbf{r}'_i = \mathbf{r}_i + D\mathbf{F}_i(\mathbf{X})\Delta t + \sqrt{2D\Delta t}\boldsymbol{\eta}, \quad D = \frac{1}{2}, \quad \boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, I).$$

The Metropolis acceptance probability is

$$A = \min\left(1, \frac{G(\mathbf{X} \rightarrow \mathbf{X}')|\Psi(\mathbf{X}')|^2}{G(\mathbf{X}' \rightarrow \mathbf{X})|\Psi(\mathbf{X})|^2}\right),$$

where

$$G(\mathbf{X} \rightarrow \mathbf{X}') = \exp\left[-\frac{\|\mathbf{X}' - \mathbf{X} - D\Delta t\mathbf{F}(\mathbf{X})\|^2}{4D\Delta t}\right].$$

b. Statistical error estimation. After parameter optimization, a long production run is performed to compute $\langle E_L \rangle$ and its statistical uncertainty. We estimate errors using the Flyvbjerg–Petersen blocking method (see Sec. III B 2).

In summary, the VMC procedure iteratively:

1. Proposes moves $\mathbf{X} \rightarrow \mathbf{X}'$ via Metropolis or importance sampling using Eqs. (16) and (11).
2. Computes $E_L(\mathbf{X})$ and $O_{\alpha_k}(\mathbf{X})$ at each accepted configuration.
3. Estimates $\langle E_L \rangle$, $\langle O_{\alpha_k} \rangle$, $\langle E_L O_{\alpha_k} \rangle$ to obtain $\partial\mathcal{L}/\partial\alpha_k$ (Eq. (13)).
4. Updates parameters $\alpha_k \rightarrow \alpha_k - \eta \partial\mathcal{L}/\partial\alpha_k$.
5. Repeats until convergence, then performs blocking analysis to estimate uncertainties.

E. Neural Networks

Feed-Forward Neural Networks and Activation Functions

A **feed-forward neural network** (FFNN) is a function approximator composed of multiple layers of interconnected neurons, where information flows in one direction, from input to output, without cycles or feedback loops. A fully connected FFNN with L layers can be expressed as:

$$\mathbf{h}^{(l+1)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{h}^{(l)} + \mathbf{b}^{(l)}\right), \quad (17)$$

where: - $\mathbf{h}^{(l)}$ is the activation vector at layer l , - $\mathbf{W}^{(l)}$ is the weight matrix connecting layers l and $l+1$, - $\mathbf{b}^{(l)}$ is the bias vector, - $\sigma(x)$ is an activation function.

The activation function introduces non-linearity, allowing neural networks to approximate complex functions. Common activation functions include:

- **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$
- **Tanh:** $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- **ReLU:** $\text{ReLU}(x) = \max(0, x)$
- **Swish:** $\text{Swish}(x) = x \cdot \sigma(x)$
- **Mish:** $\text{Mish}(x) = x \tanh(\ln(1 + e^x))$
- **GELU:** $\text{GELU}(x) = x\Phi(x)$, where $\Phi(x)$ is the standard Gaussian cumulative distribution function.

Choosing an appropriate activation function impacts model convergence and expressiveness.

Neural Network Ansatz

Using feed forward neural networks to solve the many body quantum problem is a relatively new method, and has shown very promising results. An issue with this approach, is ensuring symmetry/anti-symmetry. The structure of RBMs guarantee this, but FFNNs do not. Therefore, we need a structure that can guarantee this. FIG. 2 gives a schematic of how particle positions and pairwise distances propagate through the sub-networks to produce the correlation/jastrow factor F_θ . This structure will always be symmetric.

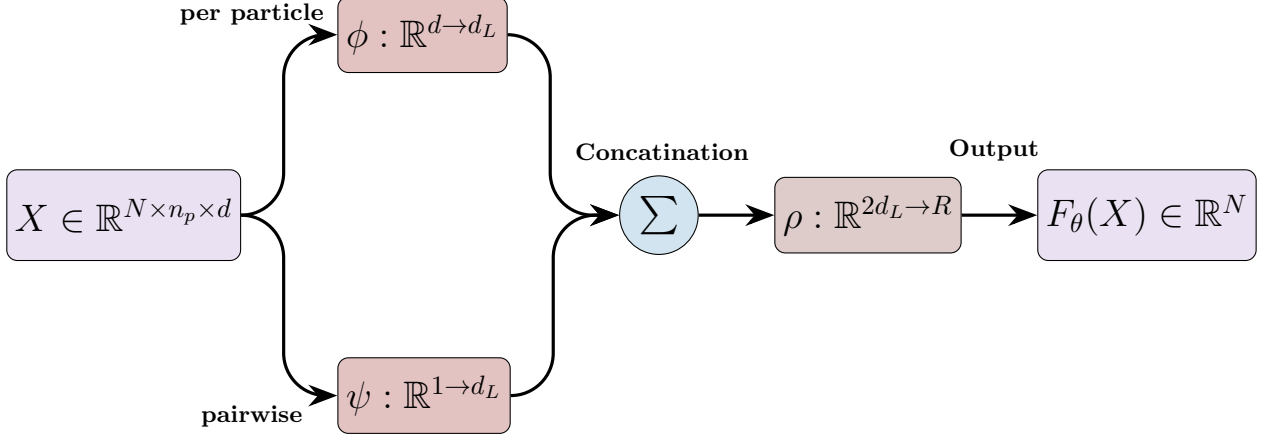


FIG. 2: Data flow in the Neural Network ansatz: batched inputs feed ϕ and ψ symmetrically into a pooling node, then through ρ to produce the Jastrow factor.

The network F_θ is realized as three sub-networks:

1. ϕ -network: $\mathbb{R}^d \rightarrow \mathbb{R}^{d_L}$, applied independently to each particle position and averaged.
2. ψ -network: $\mathbb{R} \rightarrow \mathbb{R}^{d_L}$, applied to interparticle distances $r_{ij} = |\mathbf{x}_i - \mathbf{x}_j|$.
3. ρ -network: $\mathbb{R}^{2d_L} \rightarrow \mathbb{R}$, combining the outputs of ϕ and ψ into a scalar correlation contribution.

III. METHODS

In this section we describe in detail how the RBM ansatz is implemented (Sec. III A) and how VMC is used to optimize its parameters (Sec. III B). All implementations are in Python; we outline key algorithms, data structures, and hyperparameters.

A. Restricted Boltzmann Machine

The RBM ansatz and its energy function are defined in Sec. II B, Eq. (3). In our code, we directly implement the visible-unit wave function

$$\Psi_{\text{RBM}}(X) = \sum_{\{H_j\}} e^{-E(X,H)} = \prod_{i=1}^M e^{-\frac{(X_i - a_i)^2}{2\sigma^2}} \prod_{j=1}^N (1 + e^{b_j + \sum_i X_i W_{ij}/\sigma^2}), \quad (18)$$

where $E(X, H)$ is given in Eq. (3) of Sec. II B. All parameter and variable notation follows that definition. We encapsulate this in a Python class `RBMWaveFunction` using the methods described in Appendix V, where each method directly implements the analytic formulas derived in Sec. II.

B. Variational Monte Carlo

We use **VMC** to minimize the expectation value $\langle H \rangle = \langle \Psi | \hat{H} | \Psi \rangle / \langle \Psi | \Psi \rangle$ with respect to the **RBM** parameters $\theta = (a_i, b_j, W_{ij}, \log \sigma)$. Our Hamiltonian (in atomic units, $\hbar = c = m = e = 1$) for P particles in two dimensions is

$$\hat{H} = \sum_{i=1}^P \left(-\frac{1}{2} \nabla_i^2 + \frac{1}{2} \omega^2 r_i^2 \right) + \sum_{i < j} \frac{1}{r_{ij}},$$

where $r_{ij} = |\mathbf{r}_i - \mathbf{r}_j|$. We wrap our **RBM** wavefunction inside an **InteractingWaveFunction** when including Coulomb interactions (Sec. III B 1), but first describe pure **VMC** sampling on the harmonic part. The method closely matches what we implemented in Project 1 [5].

a. Monte Carlo sampler We implement both brute-force Metropolis and importance-sampling (Langevin) moves in a class **MonteCarlo**. Additional details relating to this implementation can be found in the Appendix V.

b. Collecting samples After equilibration, we record local energies and wavefunction derivatives every n_{sample} steps. Concretely, for each accepted configuration $\mathbf{r}^{(k)}$ we compute:

$$E_L^{(k)} = \frac{1}{\Psi(\mathbf{r}^{(k)})} \hat{H} \Psi(\mathbf{r}^{(k)}) \quad O_\alpha^{(k)} = \frac{\partial \ln \Psi}{\partial \alpha}(\mathbf{r}^{(k)}),$$

for each variational parameter $\alpha \in \{a_i, b_j, W_{ij}, \log \sigma\}$. These two arrays $(E_L^{(k)}, O_\alpha^{(k)})$ are used in gradient estimates (below) and in blocking analysis (Sec. III B 2).

c. Gradient estimation and optimization At iteration t , denote by N_{MC} the number of collected samples after equilibration. We estimate

$$\langle E_L \rangle = \frac{1}{N_{\text{MC}}} \sum_{k=1}^{N_{\text{MC}}} E_L^{(k)}, \quad \langle O_\alpha \rangle = \frac{1}{N_{\text{MC}}} \sum_{k=1}^{N_{\text{MC}}} O_\alpha^{(k)},$$

$$\langle E_L O_\alpha \rangle = \frac{1}{N_{\text{MC}}} \sum_{k=1}^{N_{\text{MC}}} (E_L^{(k)} O_\alpha^{(k)})$$

The stochastic gradient of the energy with respect to each parameter α is

$$G_\alpha = 2 \left(\langle E_L O_\alpha \rangle - \langle E_L \rangle \langle O_\alpha \rangle \right). \quad (19)$$

We then update the full parameter vector θ via a chosen optimizer (SGD or Adam) using

$$\theta^{(t+1)} = \mathbf{U} \left(\theta^{(t)}, \{G_\alpha\}_\alpha \right), \quad (20)$$

where $\mathbf{U}$ is, for example:

$$\text{SGD: } \theta_i \mapsto \theta_i - \eta G_{\theta_i},$$

with learning rate η chosen between 10^{-2} and 10^{-3} for **RBM**, as tested empirically. In code, implement an abstract base class **Optimizer** and two concrete subclasses:

- **SGD(lr)**: vanilla stochastic gradient descent with step size lr .
- **Adam(lr, $\beta_1, \beta_2, \varepsilon$)**: Adaptive moment estimation; see [6].

The outer optimization loop runs for N_{iter} iterations (typically 30-100). In each iteration:

1. Update the internal **RBM** parameters by `wavefunc.update_parameters( $\theta$ )`.
2. Run N_{MC} Monte Carlo cycles to collect $\{E_L^{(k)}, O_\alpha^{(k)}\}$ after equilibration.
3. Compute $\langle E_L \rangle, \langle O_\alpha \rangle, \langle E_L O_\alpha \rangle$, then G_α via (19).
4. Call `optimize.update( $\theta \nabla E$ )` to get $\theta \leftarrow \theta - \dots$
5. Record $(\langle E_L \rangle, \text{Var}[E_L], \|\nabla E\|, \text{acceptance})$ for diagnostics.

1. Including Coulomb Interactions

In the interacting case, we wrap the **RBM** ansatz inside an **InteractingWaveFunction** object. The Coulomb potential is

$$V_{\text{Coulomb}}(\mathbf{r}_1, \dots, \mathbf{r}_P) = \sum_{i < j} \frac{1}{|\mathbf{r}_i - \mathbf{r}_j|}.$$

Concretely, one writes

$$\Psi_{\text{int}}(\mathbf{X}) = \Psi_{\text{RBM}}(\mathbf{X}) \times \exp \left[\sum_{i < j} \frac{a_{\text{jastrow}}}{r_{ij}} \right],$$

where a_{jastrow} is an optional Jastrow parameter (see Sec. III B 1, Eq. (15)). Details of how this is implemented in our code can be found in Appendix V.

2. Blocking Analysis

After finalizing the optimized parameters θ^* , we perform a long production run ($N_{\text{cycles}} = 10^6$) to estimate $\langle E_L \rangle$ with statistical uncertainty. We apply the Flyvbjerg-Petersen blocking method:

1. Takes as input an array of consecutive local-energy samples $\{E_L^{(k)}\}_{k=1}^M$.
2. At each blocking level ℓ , replaces adjacent pair by their average, producing a new array of length $\lfloor M/2 \rfloor$, and computes the sample variance Var_ℓ .
3. Continues blocking until the estimated standard error $\text{Err}_\ell = \sqrt{\text{Var}_\ell / n_\ell}$ stabilizes within tolerance.

The final result is quoted as

$$E = \frac{1}{M} \sum_{k=1}^M E_L^{(k)} \pm \text{Err}.$$

C. Neural-Network Jastrow

In place of the RBM, we model many-body correlations by a feed-forward neural network (NN) that outputs a scalar “log-Jastrow” correction. Concretely, let $X \in \mathbb{R}^{N \times D}$ be a configuration of N particles in D dimensions, and write $x_{\text{flat}} \in \mathbb{R}^{n_v}$ with $n_v = N \cdot D$. In the training procedure, we evaluate the wavefunction logarithmically, therefore we interpret the NN as

$$\ln \Psi_{\text{NN}}(X) = \text{NN}(x_{\text{flat}}).$$

The full trial wavefunction becomes

$$\Psi(X) = \ln [\det_{\uparrow}(X) \det_{\downarrow}(X)] \exp [\ln \Psi_{\text{NN}}(X)],$$

where $\det_{\uparrow}$ and $\det_{\downarrow}$ are as in Sec. III A (two 1×1 Slater blocks for spin-up/spin-down).

a. Variational Optimization. During VMC, the Slater determinants remain fixed (no trainable orbitals), so gradients of the energy with respect to Θ come solely from $\ln \Psi_{\text{NN}}(X)$. We collect a batch of local energies $\{E_{\text{loc}}^{(m)}\}$ and configurations $\{X^{(m)}\}$, then minimize the policy-gradient loss

$$\mathcal{L} = 2 \left\langle (E_{\text{loc}}(X) - \bar{E}) \ln \Psi(X; \Theta) \right\rangle,$$

with $\bar{E}$ the sample mean. Backpropagation through this expression updates all $W^{(\ell)}$, $b^{(\ell)}$. We use the Adam optimizer, learning rate 10^{-3} , and weight decay 10^{-5} , iterating for $O(10)$ epochs with $O(10^2)$ samples per epoch. Activation function σ_{act} is chosen as GELU by default; alternative smooth nonlinearities (ReLU, Tanh) also work.

D. Tools

We used the Python library `matplotlib` [7] to produce all plots in this report. We used `Numpy` [8] for mathematical calculations and arrays. `Pandas` [9] was used for formatting data using data frames and writing results to a file. We used the Adam [6] optimizer. Open AI's ChatGPT[10] was used for code debugging in our program. We also made use of ChatGPT for writing tips and suggestions in this report, such as outlining topics and rephrasing concepts for clarity.

IV. RESULTS AND DISCUSSION

A. Non-interacting System

To find the best parameters for our **RBM** we tested our non-interacting system for one particle in one dimension, where we implemented multiple parameters and optimizer variations. For this simplest test case, we observed the behavior with 10,000 cycles and 100 iterations.

We ran this test for the standard, “brute force,” metropolis and metropolis with importance sampling methods at learning rates of 0.1, 0.01, and 0.001 using the Adam optimizer. These methods were also tested for the **Stochastic Gradient Descent (SGD)** optimizer at learning rates of 0.1 and 0.01. These results, plotted in FIG. 3, found that all methods converged the quickest with the least oscillations when using a learning rate of 0.1.

We tested the Adam and **SGD** optimizers for both the brute force and importance sampling methods at this best learning rate value equal to 0.1. Comparing both methods found that in both the brute force and the importance sampling cases, the **SGD** optimizer converged the quickest with the fewest oscillations, as plotted in FIG. 4. To further reinforce these as our best parameters, we tested the brute force method versus importance sampling method, where we found including importance sampling had fewer oscillations and faster convergence.

Now that we have found the best learning rate, method, and optimizer, we used these parameters to vary the number of hidden units in our **RBM**. We tested 2, 3, 4, 5, and 10 hidden units, plotted in FIG. 5. 5 hidden units fluctuated the least, although increasing the number of hidden units did not result in drastic differences between the behavior of the system.

These best parameters for our non-interacting case; learning rate = 0.1, **SGD** optimizer, metropolis with importance sampling, are used to perform the statical analysis in section IV B. We expect the system to converge at a mean energy value of 0.5. In previous test cases the values did converge around that point, although some showed more significant fluctuations, reaching values as high as 0.9. These fluctuations are not ideal due to the nature of our system, so we identified what parameters lead to the fastest convergence at a mean energy of 0.5.

B. Blocking Analysis of Non-interacting System

Before running simulations for a full system with interactions, we ran a statistical analysis on the simple 1 particle 1 dimension non-interacting case using the Blocking method. We first found the optimal weights and biases by running the optimizer with the setup stated in section IV A and selecting the run which had the smallest variance. A plot of the variance and mean energy for the optimization run is shown in FIG. 6. Predictably, these have a similar shape and show convergence with minimal fluctuation after 15 iterations. Approximate values for the optimized parameters used for the full Monte Carlo run are

$$\begin{aligned} a &= -0.0212 \\ \mathbf{b} &= (0.100, 0.164, 0.416, 0.107, 0.038) \\ \mathbf{W} &= (-0.018, 0.243, -0.115, 0.285, -0.006). \end{aligned} \tag{21}$$

For the full run we used 1 million Monte Carlo cycles while sampling the energy from each iteration. This yields the mean energy with a blocking error of

$$E = 0.49999 \pm 0.000015. \tag{22}$$

This gives us a mean value more than close enough to the expected mean energy of 0.5, with a small error for the simplest case. To reinforce that the current setup for number of hidden units and choice of optimizer works, we ran it again but with the slightly more in depth system of 2 particles in 2 dimensions to gain a solid footing

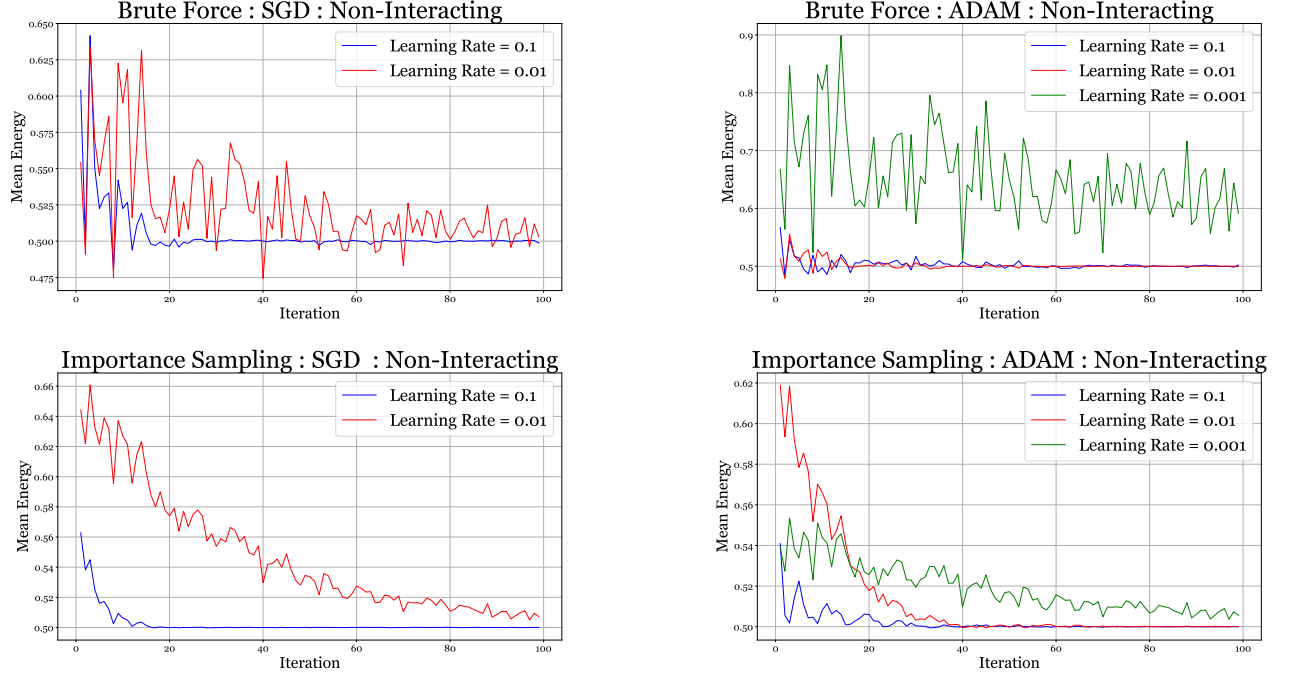


FIG. 3: Comparing Adam and **SGD** optimizers with importance sampling. Learning rates of 0.1, 0.01, and 0.001 are plotted in blue, red, and green respectively on each plot. The brute force metropolis method with **SGD** optimizer is plotted in the top left. The brute force metropolis method with Adam optimizer is plotted in the top right. The importance sampling method with **SGD** optimizer is plotted in the bottom left. The importance sampling method with Adam optimizer is plotted in the bottom right. These are all plotted for the non-interacting case.

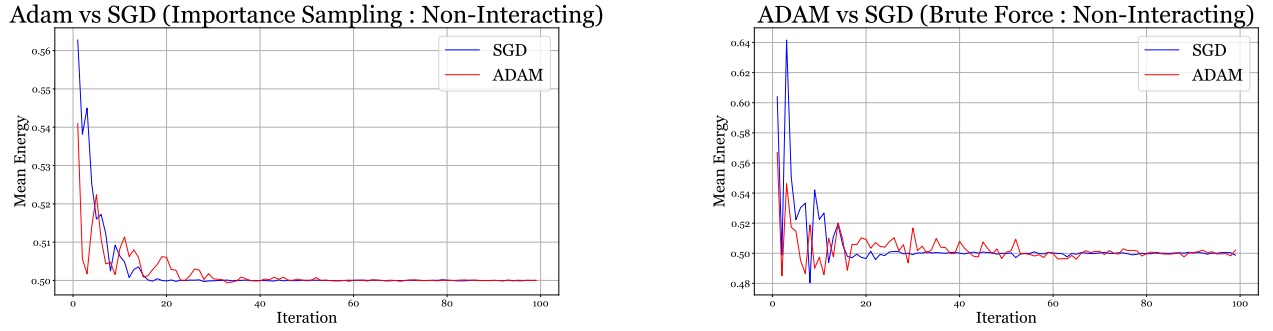


FIG. 4: Comparing the brute force metropolis(right) versus importance sampling(left) case for both the Adam(red) and **SGD**(blue) optimizers with a learning rate of 0.1 for the non-interacting case.

before implementing interactions. The variance and mean energy for the optimization run is shown in FIG. 7. This optimization shows very promising results as it converges to zero variance and the correct mean energy quicker than the previous case, using only 4 iterations. As the number of parameters has increased significantly with more particles *and* dimensions we will not be displaying them here. The iteration which gives the smallest variance is selected for a full Monte Carlo run with 1 million cycles giving us a mean energy and error of

$$E = 2.0018 \pm 0.00015. \quad (23)$$

This also matches the expected mean energy of 2 for this system, although with an error an order of magnitude bigger. However, this is to be expected as the complexity of the system has increased.

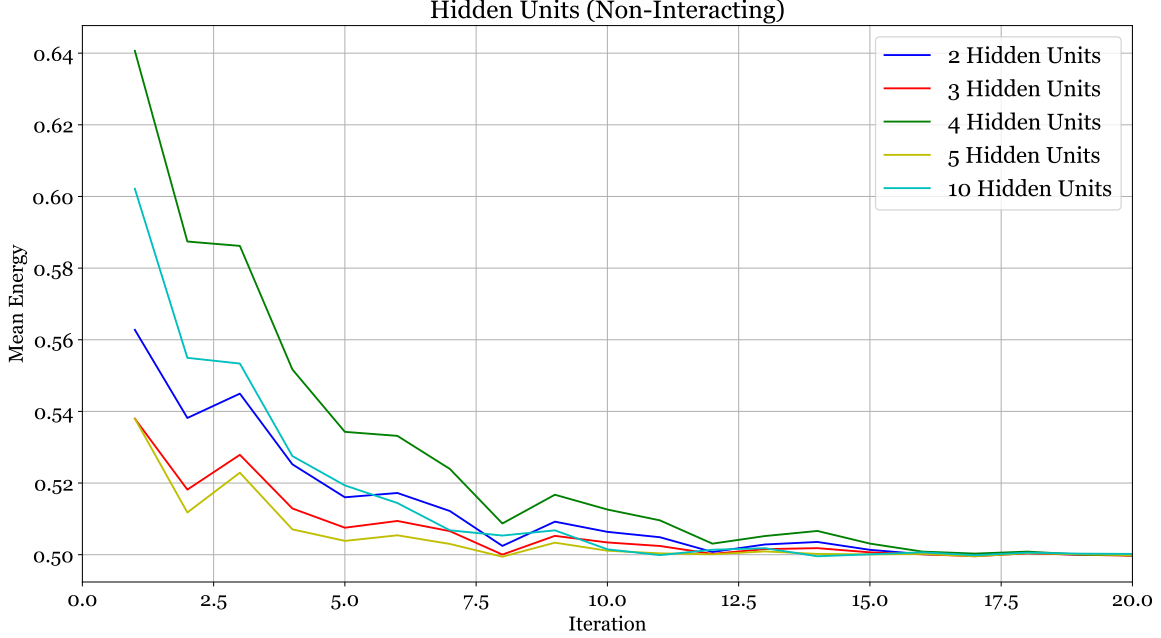


FIG. 5: Comparing number of hidden units for importance sampling with **SGD** optimizer and learning rate of 0.1. The following number of hidden units are plotted 2,3,4,5, and 10 in blue, red, green, yellow, and cyan respectively.

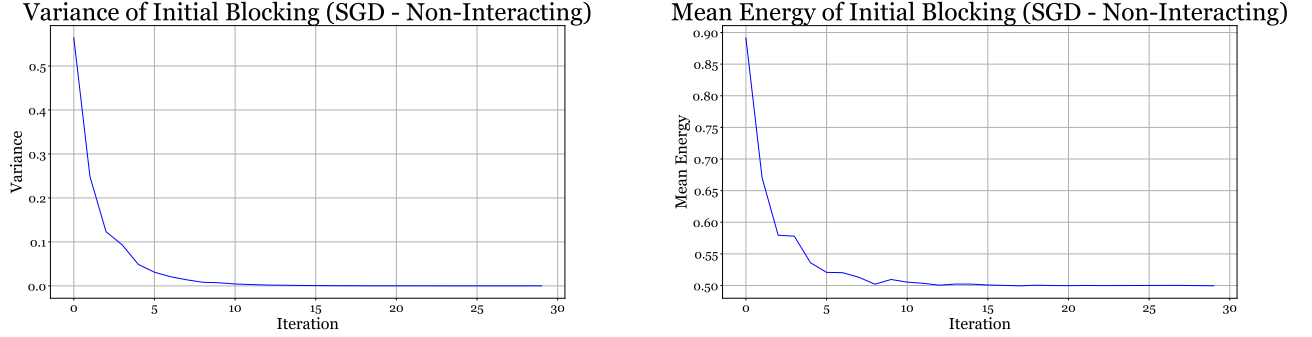


FIG. 6: Variance (left) and mean energy (right) for the optimization run of 1 particle in 1 dimension.

C. Interacting System

To look at the interacting system, we performed a similar set of test cases as done in section IV A. We introduced an additional particle and dimension, meaning the interacting system included 2 particles in 2 dimensions. Compared to the non-interacting system, the interacting system converges at a mean energy value of 3.0. In this case, we find that metropolis with importance sampling using the **SGD** optimizer had the fastest convergence with little oscillation at a learning rate of 0.01. A comparison of two learning rates for the **SGD** optimizer, and a comparison of the **SGD** to the Adam optimizer are plotted in Fig. 8. These plots are subset of the test cases performed to highlight the best combination of parameters.

We found that the same parameters worked well in both the interacting and non-interacting system, although the learning rate needed to be adjusted for the interacting system. In the interacting system a learning rate of 0.01 performed better than 0.1 (the ideal parameter in the non-interacting system). This highlights the importance of finding best parameters for the system we are evaluating, where each system may require adjustments to the parameters to provide best results.

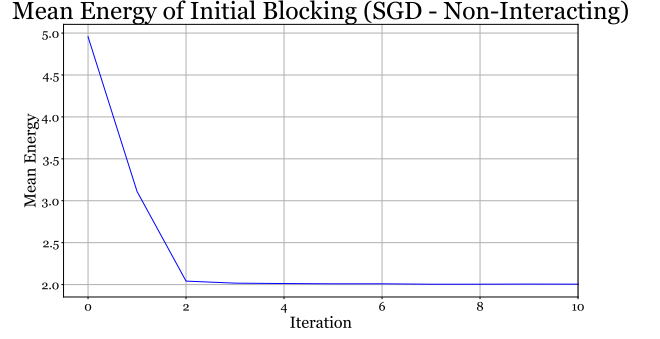
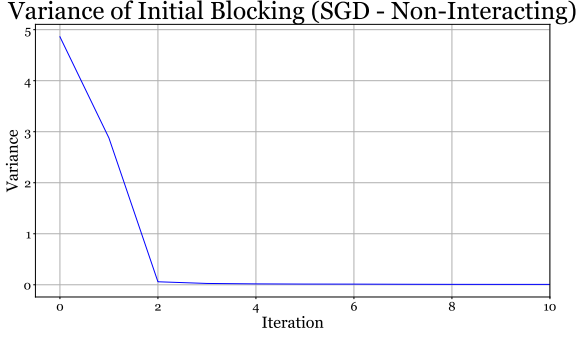


FIG. 7: Variance (left) and mean energy (right) for a run of 2 particles in 2 dimensions.

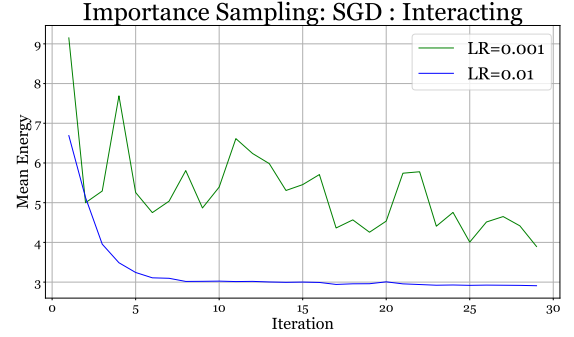
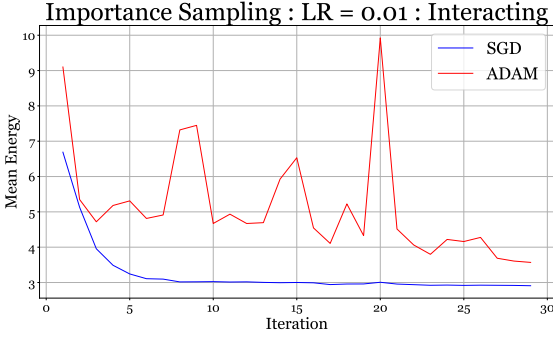


FIG. 8: Interacting system where the Adam and SGD optimizers are compared (left), and **SGD** optimizers at different learning rates are compared (right). The Adam optimizer is plotted in red while the **SGD** optimizer is plotted in blue, at a learning rate of 0.01 with importance sampling. When comparing the learning rates for the **SGD** optimizer, a learning rate of 0.001 is plotted in green, and a learning rate of 0.01 is plotted in blue.

D. Neural Network Approach

To address the limitations observed with **RBM**s in the interacting regime, we applied a feedforward **NN** ansatz to the same two-particle, two-dimensional system. Using an Adam optimizer (learning rate 1×10^{-3}) and sampling 10 000 configurations per iteration over 30 epochs, the **NN** consistently produced lower, more accurate ground-state energy estimates compared to the **RBM**. From the first epoch onward, the **NN**'s energy predictions exhibited markedly reduced fluctuations, indicating rapid stabilization (Figure 9). By epoch 30, the average variational energy obtained with the **NN** was approximately $2.32 (\pm 0.02)$, compared to the **RBM**'s converged value near 2.95. These results demonstrate that the **NN** ansatz captures interparticle correlations more effectively than the **RBM** for this interacting system.

V. CONCLUSION

In this study, we have successfully implemented a Restricted Boltzmann Machine (**RBM**) to estimate the ground state of both non-interacting and interacting particles confined in a harmonic trap via a Variational Monte Carlo (**VMC**) approach. For the single particle in one dimension, our **RBM** was trained to minimize the variational energy, and we performed a rigorous statistical analysis using the blocking method. The resulting energy estimates showed excellent agreement with the known analytical values, confirming the validity of our implementation. Upon extending to two particles in two dimensions, we found that the same network architecture and parameters yielded reliable results, provided the learning rate was increased from 1×10^{-3} to 1×10^{-2} to account for the added complexity of interparticle correlations. For this interacting system, the **RBM** converged to a mean energy of approximately 3.00, consistent with theoretical predictions.

To further improve accuracy and better capture interparticle correlations, we replaced the **RBM** ansatz with a feedforward **NN**. Employing an Adam optimizer with a learning rate of 1×10^{-3} over 30 epochs and sampling 10 000

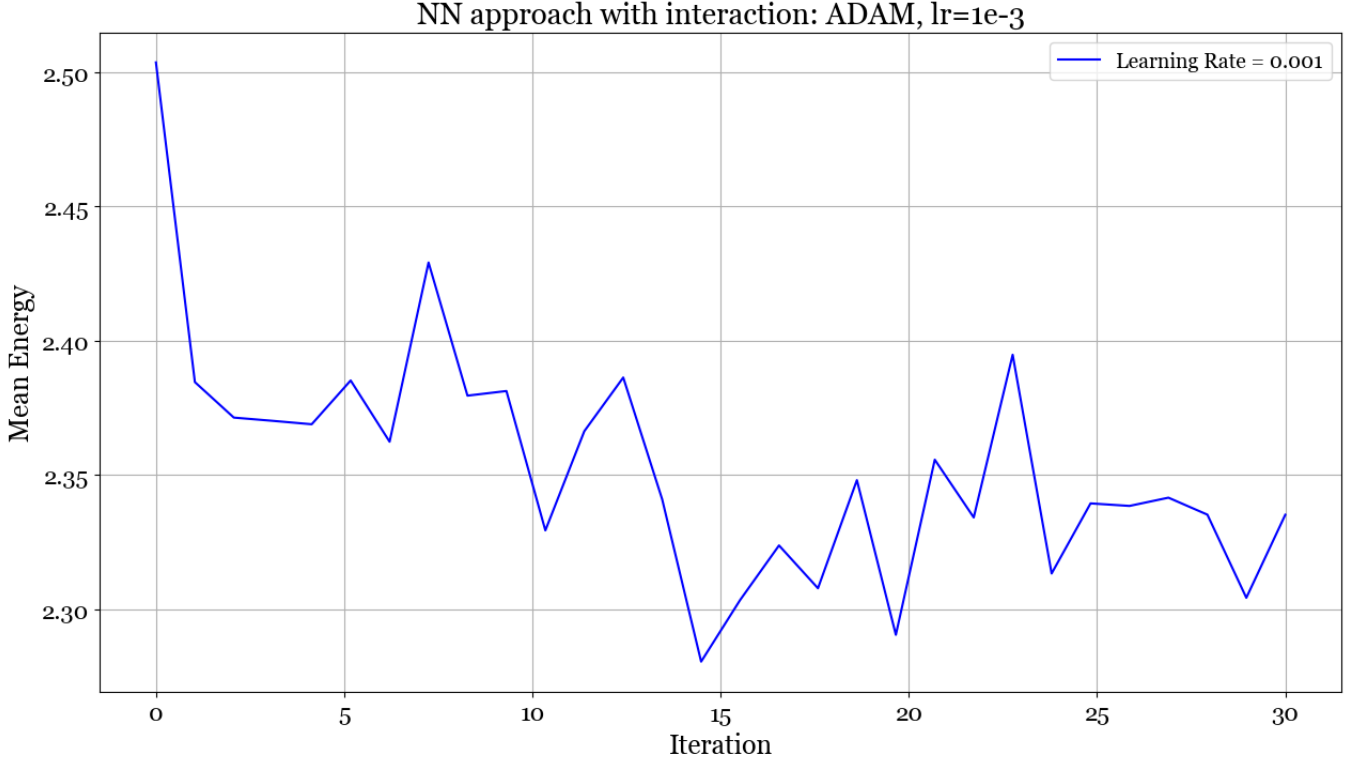


FIG. 9: Variational energies for the interacting system using the neural network ansatz with Adam optimizer ($\eta = 1 \times 10^{-3}$). Each point represents the average energy over 10 000 Monte Carlo samples; training was performed for 30 epochs.

configurations per iteration, the NN yielded lower variational energies (around 2.32 ± 0.02) and exhibited significantly reduced fluctuations from the very first epoch. These findings indicate that the NN ansatz handles correlations more effectively than the RBM in our interacting setup. By directly comparing both methods, we have highlighted that while RBMs offer a straightforward unsupervised learning framework, feedforward NNs can provide enhanced expressivity and stability for systems with nontrivial interparticle interactions.

For future work, several extensions could be pursued. First, scaling our approach to larger bosonic systems is straightforward and would allow exploration of many-body physics beyond two particles. For fermionic systems, however, it will be necessary to incorporate antisymmetry in the trial wave function—possibly by coupling RBMs or NNs with a Slater determinant or employing neural quantum states tailored to sign structure. Additionally, experimenting with deeper network architectures, convolutional layers, or alternative variational ansatz (such as transformer-based models) could further improve accuracy and convergence rates. Finally, extending the framework to accommodate different trapping potentials, higher dimensions, or external fields would provide a richer testbed for quantum many-body methods. Overall, this work demonstrates the utility of RBMs and NNs for solving quantum mechanical problems and lays the groundwork for tackling more complex systems in future studies.

-
- [1] Giuseppe Carleo and Matthias Troyer. Solving the quantum many-body problem with artificial neural networks. *Science*, 355(6325):602–606, 2017.
 - [2] Peter O’Connor, Dan Neil, Shih-Chii Liu, Tobi Delbruck, and Michael Pfeiffer. Real-time classification and sensor fusion with a spiking deep belief network. *Frontiers in Neuroscience*, 7:178, 10 2013.
 - [3] M. Taut. Two electrons in an external oscillator potential: Particular analytic solutions of a coulomb correlation problem. *Phys. Rev. A*, 48:3561–3566, Nov 1993.
 - [4] Tosio Kato. On the eigenfunctions of many-particle systems in quantum mechanics. *Communications on Pure and Applied Mathematics*, 10(2):151–177, 1957.
 - [5] William Børge Banelind, Emily Ania Puchala, Jakob Wiig Ryther, and Aleksander Sekkelsten. Studying bose-einstein condensates using variational monte carlo methods. <https://github.uio.no/jakobwr/FYS4411-VariationalMonteCarlo/>

- [blob/main/Project_1_Variational_Monte_Carlo.pdf](#), 2025. Project 1 report submitted for FYS4411: Computational Physics II at UiO available on GitHub.
- [6] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
 - [7] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
 - [8] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.
 - [9] The pandas development team. pandas-dev/pandas: Pandas, February 2020.
 - [10] OpenAI. ChatGPT (Version GPT-4) . <https://openai.com/chatgpt/>, 2024. Accessed on May 31, 2025.

APPENDIX A

A. Restricted Boltzmann Machine

Here we explain the methods used to encapsulate our **RBM** in a Python class **RBMWaveFunction** described in section III A:

1. **get_parameters()**: packs $\{a_i\}_{i=1}^M$, $\{b_j\}_{j=1}^N$, W_{ij} , and $\log(\sigma)$ into a single flat vector $\boldsymbol{\theta}$.
2. **update_parameters($\boldsymbol{\theta}$)**: unpacks the flat array back into a , b , W and $\log(\sigma)$ internally.
3. **log_psi($\mathbf{r}$)**: returns $\ln \Psi(\mathbf{X})$ given positions $\mathbf{r}$ (reshaped into $\mathbf{X} \in \mathbb{R}^M$). internally, we compute

$$\ln \Psi(\mathbf{X}) = -\frac{\|\mathbf{X} - \mathbf{a}\|^2}{2\sigma^2} + \sum_{j=1}^N \ln \left(2 \cosh \left[b_j + \frac{1}{\sigma^2} (W_j \cdot (\mathbf{X} - \mathbf{a})) \right] \right)$$

where W_j denotes row j of W .

4. **psi($\mathbf{r}$)**: returns $\Psi(\mathbf{X}) = \exp(\log_psi(\mathbf{X}))$.
5. **grad($\mathbf{r}$)**: computes

$$\frac{\partial \ln \Psi}{\partial \theta_i} \quad \text{for each variational parameter } \theta_i \in \{a_i, b_j, W_{ij}, \log \sigma\},$$

by analytic differentiation of (18). In particular, letting $\mathbf{x} = \mathbf{X} - \mathbf{a}$ and

$$\arg_j = b_j + \frac{1}{\sigma^2} W_j \cdot \mathbf{x}, \quad t_j = \tanh(\arg_j),$$

one finds:

$$\frac{\partial \ln \Psi}{\partial a_k} = \frac{x_k}{\sigma^2} - \sum_{j=1}^N t_j \frac{W_{jk}}{\sigma^2},$$

$$\frac{\partial \ln \Psi}{\partial b_j} = t_j,$$

$$\frac{\partial \ln \Psi}{\partial W_{jk}} = \frac{t_j x_k}{\sigma^2}$$

$$\frac{\partial \ln \Psi}{\partial (\log \sigma)} = \frac{\|\mathbf{x}\|^2}{\sigma^2} - \sum_{j=1}^N \frac{(W_j \cdot \mathbf{x}) t_j}{\sigma^2} \cdot \frac{2}{\sigma^2}.$$

6. **quantum_force($\mathbf{r}$)**: computes the “drift” $\mathbf{F} = 2\nabla_{\mathbf{r}} \ln \Psi$. Since $\mathbf{r} \mapsto \mathbf{X}$ is linear,

$$\mathbf{F} = 2 \left[-\frac{\mathbf{x}}{\sigma^2} + \frac{1}{\sigma^2} W^\top \mathbf{t} \right], \quad \mathbf{t} = (\tanh(\arg_1), \dots, \tanh(\arg_N))^\top, \quad \mathbf{x} = \mathbf{X} - \mathbf{a}.$$

7. **local_energy($\mathbf{r}$)**: returns the local energy

$$E_L(\mathbf{X}) = -\frac{1}{2} \left[\|\nabla_{\mathbf{X}} \ln \Psi\|^2 + \nabla_{\mathbf{X}}^2 \ln \Psi \right] + \frac{1}{2} \|\mathbf{X}\|^2,$$

where the Laplacian is computed analytically by differentiating (18). One shows

$$\nabla_{\mathbf{X}} \ln \Psi = -\frac{\mathbf{x}}{\sigma} + \frac{1}{\sigma^2} W^\top \mathbf{t}, \quad \nabla_{\mathbf{X}}^2 \ln \Psi = \frac{M}{\sigma} + \frac{1}{\sigma^4} \sum_{j=1}^N \|W_j\|^2 (1 - t_j^2),$$

with $M = 2P$ visible units.

Each method directly implements the analytic formulas derived in Sec. II.

B. Variational Monte Carlo

The following outlines the details of our the implementation of our **VMC**, where at each Monte Carlo cycle we have:

- **Brute-Force Metropolis:** Given current position $\mathbf{r}$ (an $P \times 2$ array), propose

$$\mathbf{r}' = \mathbf{r} + \delta\boldsymbol{\eta}, \quad \boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, I),$$

where δ is a tunable *step size*. Compute

$$p_{\text{old}} = \Psi(\mathbf{r})^2, \quad p_{\text{new}} = \Psi(\mathbf{r}')^2, \quad A = \min\left(1, \frac{p_{\text{new}}}{p_{\text{old}}}\right)$$

Accept $\mathbf{r}'$ with probability A ; otherwise keep $\mathbf{r}$.

- **Importance Sampling:** Compute the quantum force at $\mathbf{r}$,

$$\mathbf{F}(\mathbf{r}) = 2\nabla_{\mathbf{r}} \ln \Psi(\mathbf{r}),$$

and propose

$$\mathbf{r}' = \mathbf{r} + D\mathbf{F}(\mathbf{r})\Delta t + \sqrt{2D\Delta t}\boldsymbol{\eta}, \quad D = \frac{1}{2}, \quad \Delta t = \text{time step},$$

with $\boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, I)$. One then computes the Green's-function ratio

$$G(\mathbf{r} \rightarrow \mathbf{r}') = \exp\left[-\frac{\|\mathbf{r}' - \mathbf{r} - D\Delta t\mathbf{F}(\mathbf{r})\|^2}{4D\Delta t}\right],$$

and similarly $G(\mathbf{r}' \rightarrow \mathbf{r})$ with $\mathbf{F}(\mathbf{r}')$, to form the Metropolis acceptance

$$A = \min\left(1, \frac{G(\mathbf{r}' \rightarrow \mathbf{r})\Psi(\mathbf{r}')^2}{G(\mathbf{r} \rightarrow \mathbf{r}')\Psi(\mathbf{r})^2}\right).$$

Accept or reject accordingly. The time step Δt is tuned to achieve $\sim 80\%$ acceptance.

All random numbers are drawn with Numpy's **Generator** for reproducibility.

To implement interactions for our system, we do the following in our code:

- `InteractingWaveFunction.get_parameters()`: returns base **RBM** parameters $\oplus \{a_{\text{jastrow}}\}$ is enabled.
- `log_psi(r)`: returns

$$\ln \Psi_{\text{RBM}}(\mathbf{X}) + \sum_{i < j} \frac{a_{\text{jastrow}}}{r_{ij}}.$$

- `local_energy(r)`: returns unconditional $E_L^{\text{RBM}}(\mathbf{r}) + \sum_{i < j} \frac{1}{r_{ij}}$.
- `quantum_force(r)`: adds

$$\mathbf{F}_{\text{J}}(i) = -a_{\text{jastrow}} \sum_{j \neq i} \frac{\mathbf{r}_i - \mathbf{r}_j}{\|\mathbf{r}_i - \mathbf{r}_j\|^3}$$

to the base **RBM** drift $\mathbf{F}_{\text{RBM}}$, then multiplies by 2.

- `grad(r)`: appends

$$\frac{\partial \ln \Psi_{\text{int}}}{\partial a_{\text{jastrow}}} = \sum_{i < j} \frac{1}{r_{ij}}$$

to the gradient from the base **RBM**.

Using this wrapper, the outer **VMC** loop proceeds identically, but with $\hat{H}$ now including V_{Coulomb} .